



Module 2 – Practice Questions

Q1. In one sentence, what problem does a perceptron solve?

A **perceptron solves binary classification problems** by separating data into two classes using a decision boundary.

Q2. What are the three main components of a perceptron?

The three main components are:

1. **Inputs (x)** – the features given to the perceptron.
2. **Weights (w) and Bias (b)** – determine the importance of each input and shift the decision boundary.
3. **Activation function** – usually a **step function**, which produces the final output.

Q3. Is perceptron a classifier or a regressor? Explain briefly.

A **perceptron is a classifier**, specifically a **binary classifier**. It uses a step activation function to output one of two classes, usually **0 or 1**, based on whether the weighted sum crosses a threshold.

Q4. Why do we need to update weights in a perceptron?

We update the **weights** so that the perceptron can **learn from its mistakes** and gradually make more correct predictions.

Q5. What happens if we never update the weights during training?

If we never update the weights, the perceptron **cannot learn** from the training data, so its predictions will remain based on the initial/random weights.

Q6. Which two quantities decide how much the weight changes?

The amount of weight change depends on:

1. **Learning rate (η)** – controls how large the update is.
 2. **Error (target – predicted output)** – determines how much correction is needed.
-

Q7. What does $(y - \hat{y})$ represent in perceptron learning?

$(y - \hat{y})$ represents the **error** or **prediction error**—the difference between the actual output y and the predicted output $\hat{y}$.

Q8. If the prediction is correct, will weights change? Why?

No, the weights will not change.

Q9. What role does the learning rate (η) play intuitively?

The **learning rate controls how big each weight update is**.

- **Large η** → bigger changes → faster but potentially unstable learning.
 - **Small η** → smaller changes → slower but more gradual learning.
-

Q10. Write the AND gate truth table.

Q11. Why is AND gate linearly separable?

The AND gate is **linearly separable** because we can draw a **single straight line** that separates the output-1 point **(1,1)** from all the output-0 points **(0,0), (0,1), and (1,0)**.

Q12. Draw a rough decision boundary that separates AND gate outputs.

Q13. Why do we add a bias term in the perceptron?

The **bias allows the perceptron to shift the decision boundary**, so the boundary does not have to pass through the origin.

Q14. Why is bias often added **after summation**

bias can shift the total activation **without depending on the input values**.

Q15. Why do we initialize weights with small random values?

We initialize weights with **small random values** so that the perceptron starts from a reasonable initial point and can **learn by adjusting the weights during training**.

Small values help avoid unnecessarily large initial activations, while randomness prevents all weights from starting identically.

Q16. What is the purpose of `np.dot(X, w)` in perceptron code?

Q17. Why is matrix transpose used in weight updating?

Transpose converts the data matrix into the proper orientation so that errors from all samples can be combined to update each weight correctly.

Q18. What will happen if we remove the activation function?

If we remove the activation function, the perceptron cannot directly produce its binary class output; it becomes essentially a linear score rather than a binary classifier.

Q19. Why can OR gate be solved using a single perceptron?

Q20. What is common between AND and OR gates from a geometry view?

Q21. Does XOR need more features or more layers? Why?

The two classes are arranged diagonally, so **no single straight line can separate them**. Therefore, a single perceptron cannot solve XOR. A **multi-layer neural network** can solve XOR by creating multiple linear boundaries and combining them.

Q22. What does *linearly separable* mean?

Q23. Why can't a single straight line separate XOR data?

Q24. What change is required to solve XOR successfully?

Add layer

Q25. List two strengths of a perceptron.

1. Simple and easy to implement — the perceptron has a simple structure and learning rule.
2. Fast and computationally inexpensive — it can train quickly on simple classification problems.

Q26. List two limitations of a perceptron.

1. Cannot solve non-linearly separable problems, such as XOR.
2. Can only create a linear decision boundary, so it cannot handle complex patterns by itself.

Q27. When should we avoid using a single-layer perceptron?

We should avoid a single-layer perceptron when the data is not linearly separable or when the classification problem requires a complex/non-linear decision boundary.



Mini Challenge (Very Easy)

Q28.

Given:

- $x_1 = 1, x_2 = 1$
- weights = [1, 1]
- bias = -1.5

Will the perceptron output be **1** or **0**?

(Show reasoning in one line)

Q28. Answer: 1

One-line reasoning:

$$z = (1 \times 1) + (1 \times 1) - 1.5 = 0.5$$

Since $z \geq 0$, the step activation function gives:

$$\hat{y} = 1$$